

1. Overview of Data Governance and Unity Catalog?

Data Governance is a comprehensive framework that defines how data is collected, stored, processed, secured, and used within an organization. In modern cloud data ecosystems such as Databricks, data governance is critical because data is distributed across multiple storage layers, teams, pipelines, and analytics systems.

Without governance, organizations face major challenges such as:

- Data duplication across teams
- Unauthorized access to sensitive data
- Lack of consistency in reporting
- No visibility into data lineage
- Compliance risks (GDPR, HIPAA, etc.)

Data Governance solves these issues by introducing structured control over data assets.

Core Components of Data Governance:

1. Data Security

Ensures that only authorized users can access data. Security is enforced using role-based access control (RBAC), attribute-based access control (ABAC), and fine-grained permissions.

2. Data Quality

Ensures data is accurate, consistent, complete, and reliable. This includes validation rules, cleansing processes, and monitoring pipelines.

3. Data Lineage

Tracks how data moves and transforms across systems—from ingestion to final reporting. It helps in debugging and compliance audits.

4. Metadata Management

Stores information about datasets such as schema, owner, source, and usage history.

5. Compliance & Auditability

Ensures that all data usage is trackable and meets regulatory requirements.

Unity Catalog in Databricks

Unity Catalog is Databricks' centralized governance layer designed to manage all data assets across multiple workspaces in a unified manner. It replaces fragmented workspace-level metastore systems with a centralized architecture.

Unity Catalog is not just a metadata store—it is a complete governance system.

Key Capabilities of Unity Catalog:

1. Centralized Governance

- One governance layer for all Databricks workspaces
- Eliminates inconsistent permissions across environments

2. Fine-Grained Access Control

Supports multiple levels of security:

- Catalog-level permissions
- Schema-level permissions
- Table-level permissions
- Column-level security
- Row-level filtering

3. Data Lineage Tracking

Automatically tracks:

- Source tables
- Transformations
- Downstream datasets

This helps in impact analysis and debugging pipelines.

4. Unified Data Discovery

Users can search and access datasets across all workspaces using a single catalog interface.

5. Cross-Workspace Sharing

Multiple workspaces can use the same governed data layer without duplicating data.

Architecture Insight:

Unity Catalog sits between:

- Data storage (ADLS / S3 / GCS)
- Databricks compute clusters
- User workspaces

It acts as a governance control plane.

2. Create Unity Catalog Metastore and Enable a Databricks Workspace with Unity Catalog

A Unity Catalog Metastore is the highest-level container in Databricks governance architecture. It stores metadata for all catalogs, schemas, tables, views, and access policies. It acts as the root of the entire governance hierarchy.

Each metastore is region-specific and must be linked to cloud storage for storing managed data assets.

Pre-requirements:

Before creating a metastore:

- A Databricks account-level admin is required
- A cloud storage account must be available (ADLS / S3 / GCS)
- Proper IAM roles must be configured
- Workspace must be in supported region

Step-by-Step Process

STEP 1: Create Cloud Storage Root

This is the base storage location where managed tables are stored.

Example:

- Azure: `abfss://unitycatalog@storageaccount.dfs.core.windows.net/`
- AWS: `s3://unity-catalog-root/`

This location is used for:

- Managed Delta tables
- Metadata storage support

- Temporary system files

STEP 2: Create Metastore in Databricks Account Console

You navigate to:

Databricks Account Console → Data Governance → Unity Catalog

Then:

- Click Create Metastore
- Provide:
 - Name (e.g., `global\_metastore`)
 - Region (must match workspace region)
 - Storage root location

Once created, the metastore becomes the central metadata brain of the platform.

STEP 3: Assign Metastore to Workspace

- A workspace is linked to only one metastore at a time
- Admin assigns metastore to workspace via account console
- This step enables Unity Catalog in that workspace

After assignment:

- Workspace switches from Hive metastore → Unity Catalog
- All new data objects follow Unity Catalog rules

STEP 4: Configure Access Permissions

Admins configure:

- Storage credentials
- External locations
- IAM roles
- Service principals

This ensures secure access to cloud storage.

STEP 5: Enable Unity Catalog Usage

Once attached:

- Users can create catalogs
- Data is governed centrally

- Permissions are enforced automatically
- Lineage tracking becomes active

Impact After Setup

After enabling Unity Catalog:

Before (Old Model):

- Each workspace had its own metastore
- No centralized governance
- Hard to manage permissions

After (Unity Catalog):

- Central governance across all workspaces
- Consistent security model
- Better scalability and control

3. Overview of 3-Level Namespace and Creating Unity Catalog Objects

The 3-level namespace in Unity Catalog is a **structured hierarchical model** used to organize all data assets in Databricks. It ensures that data is logically grouped, easily accessible, and securely managed.

The structure is:

Catalog → Schema → Table/View

This hierarchy replaces traditional flat database structures and introduces enterprise-grade data organization.

1. CATALOG (Level 1 – Top Layer)

A Catalog is the highest-level container in Unity Catalog. It represents a business domain or data domain.

Examples:

- ``sales_catalog``
- ``hr_catalog``
- ``finance_catalog``

Purpose:

- Separates data by business function
- Acts as a security boundary
- Controls top-level access

Real-world analogy:

Catalog = Department (HR, Sales, Finance)

2. SCHEMA (Level 2 – Database Layer)

A Schema is a logical grouping inside a catalog.

Examples:

- `sales\_catalog.online\_sales`
- `sales\_catalog.offline\_sales`

Purpose:

- Organizes datasets into logical groups
- Represents sub-domains
- Improves data discoverability

Real-world analogy:

Schema = Teams inside department

3. TABLE / VIEW (Level 3 – Data Layer)

This is the actual data storage layer.

Examples:

- `sales\_catalog.online\_sales.transactions`
- `sales\_catalog.online\_sales.customers`

Purpose:

- Stores structured data
- Used for analytics and reporting
- Supports Delta Lake features

Creating Unity Catalog Objects (Step-by-Step)

STEP 1: Create Catalog

```
CREATE CATALOG sales_catalog;
```

This creates the top-level container.

STEP 2: Create Schema

```
CREATE SCHEMA sales_catalog.retail_schema;
```

This organizes datasets within catalog.

STEP 3: Create Table

```
CREATE TABLE sales_catalog.retail_schema.sales_transactions (  
    transaction_id INT,  
    product_name STRING,  
    quantity INT,  
    price DOUBLE,  
    sale_date DATE  
);
```

This defines structured dataset.

STEP 4: Insert Data

```
INSERT INTO sales_catalog.retail_schema.sales_transactions  
VALUES (1, 'Laptop', 2, 55000, '2026-06-18');
```

STEP 5: Grant Permissions

```
GRANT SELECT ON TABLE sales_catalog.retail_schema.sales_transactions  
TO `analyst_group`;
```

Advanced Concepts

1. Managed vs External Tables

- Managed tables → stored in Unity Catalog storage root
- External tables → stored in external cloud locations

2. Delta Lake Integration

Unity Catalog works natively with Delta tables:

- ACID transactions
- Time travel
- Schema enforcement

3. Column-Level Security Example

```
GRANT SELECT (transaction_id, product_name)
ON TABLE sales_catalog.retail_schema.sales_transactions
TO `user_group`;
```

4. Row-Level Security Example

Using row filters:

```
CREATE ROW FILTER sales_filter
AS (region = 'India');
```